

 iamSaikrishna143 / Last_Believe



[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security](#)



main ▾

Last_Believe / Sort&Searching.js



iamSaikrishna143 all update

ce579bf · 9 hours ago



183 lines (150 loc) · 3.99 KB

Code

Blame



Raw



```
1 // Bubble Sort
2 function BubbleSort(arr) {
3     let n = arr.length;
4     for (let i = 0; i < n - 1; i++) {
5         for (let j = 0; j < n - 1 - i; j++) {
6             if (arr[j] > arr[j + 1]) {
7                 let temp = arr[j];
8                 arr[j] = arr[j + 1];
9                 arr[j + 1] = temp;
10            }
11        }
12    }
13    return arr;
14 }
15 console.log(BubbleSort([10, 5, 6, 8, 4, 0]));
16
17 // -----
18
19 // Selection Sort - Asc order
20 function selectionSort(arr) {
21     let n = arr.length;
22
23     for (let i = 0; i < n - 1; i++) {
24         // Assume the current index has the smallest element
25         let minIndex = i;
26
27         // Find the index of the smallest element in the remaining array
28         for (let j = i + 1; j < n; j++) {
29             if (arr[j] < arr[minIndex]) {
30                 minIndex = j;
31             }
32         }
33
34         // Swap if a smaller element is found
35         if (minIndex !== i) {
36             let temp = arr[i];
```

```
37     arr[i] = arr[minIndex];
38     arr[minIndex] = temp;
39 }
40 }
41
42 return arr;
43 }
44
45 console.log(selectionSort([10, 5, 6, 8, 4, 0]));
46
47 // -----
48 // Desc-order
49 ✓ function selectionSortDescending(arr) {
50     let n = arr.length;
51
52     for (let i = 0; i < n - 1; i++) {
53         // Assume the current index has the largest element
54         let maxIndex = i;
55
56         // Find the index of the largest element in the remaining array
57         for (let j = i + 1; j < n; j++) {
58             if (arr[j] > arr[maxIndex]) {
59                 maxIndex = j;
60             }
61         }
62
63         // Swap if a larger element is found
64         if (maxIndex !== i) {
65             let temp = arr[i];
66             arr[i] = arr[maxIndex];
67             arr[maxIndex] = temp;
68         }
69     }
70
71     return arr;
72 }
73
74 console.log(selectionSortDescending([10, 5, 6, 8, 4, 0]));
75
76 // Insertion Sort- ASC order
77 ✓ function insertionSort(arr) {
78     let n = arr.length;
79     for (let i = 1; i < n; i++) {
80         let key = arr[i]; // element to be inserted
81         let j = i - 1;
82         // Move elements greater than key one position ahead
83         while (j >= 0 && arr[j] > key) {
84             arr[j + 1] = arr[j];
85             j--;
86         }
87         arr[j + 1] = key;
88     }
```

```
89     return arr;
90 }
91
92 console.log(insertionSort([10, 5, 6, 8, 4, 0]));
93
94 // -----
95 ✓ function insertionSortDescending(arr) {
96     let n = arr.length;
97
98     for (let i = 1; i < n; i++) {
99         let key = arr[i];
100        let j = i - 1;
101
102        // Move elements smaller than key one position ahead
103        while (j >= 0 && arr[j] < key) {
104            arr[j + 1] = arr[j];
105            j--;
106        }
107
108        arr[j + 1] = key;
109    }
110
111    return arr;
112 }
113
114 console.log(insertionSortDescending([10, 5, 6, 8, 4, 0]));
115
116 // GCD Check using recursion
117 ✓ function gcd(a, b) {
118     if (a == b) return a;
119     if (a > b) return gcd(a - b, b);
120     return gcd(b, b - a);
121 }
122 console.log(gcd(32, 16));
123
124 // /-----
125
126 // Binary Search
127 let arr = [2, 5, 8, 9, 12, 25, 68, 100, 150, 200, 250];
128 let index = binarySearch(arr, 0, arr.length - 1, 68);
129 if (index == -1) console.log("not found");
130 else console.log("target found" + index + "index");
131
132 ✓ function binarySearch(arr, first, last, target) {
133     while (first <= last) {
134         let mid = Math.floor((first + last) / 2);
135         if (arr[mid] == target) return mid;
136         else if (arr[mid] > target) last = mid - 1;
137         else first = mid + 1;
138     }
139     return -1;
140 }
```

```
141
142 // -----
143
144 // Merge Sort
145
146 ✓ function conquer(arr, first, mid, last) {
147     let temp = new Array(last - first + 1);
148     let i = first,
149         j = mid + 1,
150         k = 0;
151
152     while (i <= mid && j <= last) {
153         if (arr[i] < arr[j]) {
154             temp[k++] = arr[i++];
155         } else temp[k++] = arr[j++];
156     }
157     while (i <= mid) {
158         temp[k++] = arr[i++];
159     }
160     while (j <= last) {
161         temp[k++] = arr[j++];
162     }
163     let p = 0,
164         t = first;
165     while (p < temp.length) {
166         arr[t++] = temp[p++];
167     }
168 }
169
170 ✓ function divide(arr, first, last) {
171     if (first >= last) return;
172     let mid = Math.floor((first + last) / 2);
173     divide(arr, first, mid);
174     divide(arr, mid + 1, last);
175     conquer(arr, first, mid, last);
176 }
177
178 arr = [1, 2, 8, 5, 15, 8, 5, 22, 3, 55, 6];
179 divide(arr, 0, arr.length - 1);
180 console.log(arr);
181
182
183 -----
```